Experiment 2

Working with Maven: Creating a Maven Project, Understanding the POM File, Dependency Management and Plugins

Steps to Install IntelliJ Idea

1️⃣ Download

- Go to 👉 https://www.jetbrains.com/idea/download

2️⃣ Run Setup

- Open downloaded file (ideaIC.exe)
- Click Next

3️⃣ Select Options

✔ Create Desktop Shortcut
✔ Update PATH
✔ Associate .java files
Click Next → Install

4️⃣ Finish & Launch

- Click Finish
- Open IntelliJ IDEA
- Select Do not import settings

Steps to Create a Maven Project in IntelliJ IDEA

1. **Install Maven**:

- Download Maven from the official website.
- Set the **MAVEN_HOME** environment variable and update the system **PATH**.

2. **Create a New Maven Project**:

- Open IntelliJ IDEA.
- Go to File > New > Project.
- Give a name to the project.
- Select Maven option in the build system.
- Choose Create from Archetype (optional) or proceed without.
- Set the project name and location, then click Finish.

3. Set Up the pom.xml File:

- The pom.xml file is where you define dependencies, plugins, and other configurations for your Maven project.

Example of a basic pom.xml :

```
<project xmlns=http://maven.apache.org/POM/4.0.0
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
        <modelVersion>4.0.0</modelVersion>
        <groupId>com.example</groupId>
        <artifactId>simple-project</artifactId>
        <version>1.0-SNAPSHOT</version>
        <dependencies>
                <!-- Add your dependencies here -->
        </dependencies>
</project>
```

4. Add Dependencies for Selenium and TestNG:

- In the pom.xml, add Selenium and TestNG dependencies under the <dependencies> section.

```
<dependencies>
    <dependency>
            <groupId>org.seleniumhq.selenium</groupId>
        <artifactId>selenium-java</artifactId>
        <version>3.141.59</version>
    </dependency>
    <dependency>
            <groupId>org.testng</groupId>
        <artifactId>testng</artifactId>
        <version>7.4.0</version>
        <scope>test</scope>
    </dependency>
    </dependencies>
```

5. Create a Simple Website (HTML, CSS, and Logo):

- In the src/main/resources folder, create an index.html file, a style.css file, and place the logo.png image.

Sample index.html file:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>My Simple Website</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <header>
            <img src="logo.png" alt="Logo">
    </header>
            <h1>Welcome to My Simple Website</h1>
</body>
</html>
```

simple style.css:

```
body {
    font-family: Arial, sans-serif;
    background-color: #f4f4f4;
    text-align: center;
}
header img {
    width: 100px;
}
```

6. Upload the Website to GitHub:

- Initialize a Git repository in your project folder:

  ```
  git init
  ```

- Add your files and commit them:

  ```
  git add .
  git commit -m "Initial commit"
  ```

- Create a GitHub repository and push the local project to GitHub:

  ```
  git remote add origin <your-repository-url>
  git push -u origin master
  ```

Deployment:

To deploy your Maven project to **GitHub Pages** using the /docs folder (**** having all files inside root folder/dir not recommended**), you can follow these simple steps. This method is easy and doesn't require switching branches—just use the /docs folder of your main branch.

Steps to Deploy to GitHub Pages Using /docs **Folder**

1. **Modify Maven Configuration to Copy Static Files to** /docs **Folder**: First, you need to ensure that Maven places your static files (index.html , style.css , logo.png) into the /docs folder instead of the target directory.

To do this, configure the **Maven Resources Plugin** in your pom.xml to copy the files directly into /docs :

```
<build>
    <plugins>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-resources-plugin</artifactId>
            <version>3.2.0</version>
<executions>
    <execution>
        <phase>prepare-package</phase> <!-- Before packaging ->
<goals>
    <goal>copy-resources</goal>
</goals>
<configuration>
    <outputDirectory>${project.basedir}/docs</outputDirectory> <!-- Deploy to
/docs
folder -->
<resources>
    <resource>
        <directory>src/main/resources</directory>
<includes>
    <include>**/*</include> <!-- Copy all files in src/main/resources -->
</includes>
    </resource>
</resources>
</configuration>
    </execution>
</executions>
</plugin>
</plugins>
</build>
```

2. **Build the Project**: Run the following Maven command to build your project and copy the resources to the /docs folder:

```
mvn clean install
```

3. Push Changes to GitHub: Now that the files are in the /docs folder, they are ready to be served by GitHub Pages. Follow these steps:

a) Stage the changes (the updated /docs folder):

```
git add docs/*
git commit -m "Deploy site to GitHub Pages"
```

b) Push to GitHub:

```
git push origin master
```

4. Enable GitHub Pages: After pushing to the main branch, follow these steps to enable GitHub Pages: Go to your GitHub repository.
Navigate to **Settings** > **Pages** (on the left sidebar).
Under the **Source** section, select the main branch and /docs folder as the source.
Click **Save**.

5. Access Your Website: Your static website is now hosted on GitHub Pages! You can access it at:

https://<your-github-username>.github.io/<your-repository-name>/

6. **Write a Simple Selenium Test with TestNG**:

Create a new Java class WebPageTest.java in the src/test/java directory.
Example of a simple TestNG test using Selenium:

```
package org.test;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.testng.Assert;
import org.testng.annotations.AfterTest;
import org.testng.annotations.BeforeTest;
import org.testng.annotations.Test;

import static org.testng.Assert.assertTrue;

public class WebpageTest {
private static WebDriver driver;

@BeforeTest
public void openBrowser() throws InterruptedException {
        driver = new ChromeDriver();
        driver.manage().window().maximize();
        Thread.sleep(2000);
        driver.get("https://sauravsarkar-codersarcade.github.io/CA-MVN/");  //  "Note: You should use your GITHUB-URL here...!!!"
        }
```

```
    @Test
    public void titleValidationTest(){
        String actualTitle = driver.getTitle();
        String expectedTitle = "Tripillar Solutions";
        Assert.assertEquals(actualTitle, expectedTitle);
        assertTrue(true, "Title should contain 'Tripillar'");
    }

    @AfterTest
    public void closeBrowser() throws InterruptedException {
        Thread.sleep(1000);
        driver.quit();
    }
}
```

7. **Run the Test**:
In IntelliJ, right-click the WebPageTest class and select Run 'WebPageTest'.
The test will launch Chrome, open the webpage, and validate the title.